

From Static Prompts to Dynamic Orchestration: A Synthesis of Evolution, Optimization, and Security in LLM Tool Learning

The integration of Large Language Models (LLMs) with external tools has catalyzed a paradigm shift from static, prompt-dependent generators to dynamic, goal-directed agents. This synthesis traces the field's evolution from brittle in-context learning to robust, policy-driven frameworks utilizing reinforcement learning for multi-step planning [99]. We analyze the transition toward dynamic orchestration architectures that decouple retrieval and execution, addressing the "semantic-functional gap" where textual relevance fails to ensure operational viability [42]. Furthermore, we examine emerging optimization strategies, including compiler-based skill compilation and speculative execution, which mitigate latency and token overhead in enterprise-scale deployments [251]. However, this increased autonomy introduces critical vulnerabilities, such as Time-of-Check to Time-of-Use (TOCTOU) attacks and stealthy resource amplification, necessitating a re-evaluation of security beyond single-turn validation [296]. Finally, we highlight the urgent need for comprehensive benchmarks that assess tool necessity, stateful dependencies, and the "knowing-doing gap" in complex, interdependent environments [262]. This review underscores that future advancements depend on balancing adaptive self-evolution with rigorous economic and safety constraints.

Introduction to Tool Learning and Function Calling Paradigms

The integration of Large Language Models (LLMs) with external tools has catalyzed a fundamental paradigm shift, transforming static text generators into dynamic, goal-directed agents capable of interacting with complex environments. This evolution marks a transition from relying solely on parametric knowledge to leveraging non-parametric resources, thereby addressing inherent limitations such as factual hallucinations, outdated information, and computational inaccuracies [99]. The foundational concept of tool learning posits that LLMs can augment their reasoning and execution capabilities by invoking external APIs, code interpreters, and search engines, effectively bridging the gap between linguistic understanding and real-world action [38]. Early explorations in this domain, such as Retrieval-Augmented Generation (RAG) and neuro-symbolic approaches, laid the groundwork for modern agentic systems by demonstrating the value of coupling generative models with external memory and symbolic solvers [99]. However, the field has rapidly progressed beyond simple retrieval, moving toward sophisticated architectures that support multi-step planning, autonomous tool creation, and self-improvement loops [153].

A critical distinction in the current literature lies in the methodological evolution from prompt-based interactions to policy-driven learning. Initial approaches treated tool use as a "plug-and-play" capability, relying on frozen model parameters guided by in-context learning and intricate prompt engineering [99]. While flexible, these methods often suffer from brittleness in complex, multi-step scenarios and high latency due to extensive token generation [99]. To overcome these bottlenecks, the field has shifted toward Supervised Tool Learning, where models are fine-tuned on high-quality datasets to internalize the syntax and logic of tool invocation [99]. Frameworks like ToolLLM and ToolBench have been instrumental in this regard, constructing large-scale instruction-tuning datasets that enable open-source models to master thousands of real-world APIs, narrowing the performance gap with closed-source counterparts [12]. Furthermore, recent advancements in data synthesis, such as TOOLFLOW and GENESISFUNC, emphasize the necessity of coherent, multi-turn dialogue generation to ensure training data reflects the nuances of real-world tool usage [218] [280]. These data-centric approaches are complemented by automated pipelines like ToolACE, which curate diverse API pools to generate accurate and complex training instances, further enhancing the generalizability of tool-augmented models [191].

Despite the successes of supervised fine-tuning, treating tool learning merely as a text-generation task overlooks the sequential decision-making complexities inherent in agentic workflows. Consequently, a third paradigm, Reward-Driven Tool Policy Learning, has emerged, framing tool use as a dynamic optimization problem solvable through reinforcement learning (RL) [99]. Methods like StepTool introduce step-grained reward shaping to optimize multi-step tool usage, addressing the sparse reward problem that plagues traditional RL approaches in this domain [139]. Similarly, frameworks such as ToolMaster and RLTR decouple planning from execution or utilize trial-and-execution paradigms to enhance robustness against novel tools and unclear instructions [286] [178]. These approaches highlight a consensus that while supervised learning provides a strong baseline, reinforcement learning is essential for developing agents capable of long-horizon planning and adaptive strategy selection [281].

Significant challenges remain, particularly regarding the disparity between open-source and closed-source models and the scalability of tool integration. Open-source models often struggle with the sheer volume of available tools and the precision required for parameter filling, leading to research focused on efficient tool selection and interface optimization [49] [18]. Innovations like AutoTool and EcoAct address inference costs by leveraging statistical patterns or selective tool registration, reducing the computational overhead associated with repeated LLM calls for tool selection [16] [115]. Moreover, the definition and standardization of "tools" vary across studies, prompting calls for unified taxonomies and evaluation benchmarks that assess not only function-call correctness but also end-to-end task success in interactive, stateful environments [37] [100]. The transition from reactive prompting to autonomous agency also necessitates addressing the "knowing-doing gap," where models recognize the need for a tool but fail to execute the call correctly, a phenomenon recently identified through model-adaptive necessity evaluations [266]. As the field matures, the focus is increasingly shifting toward creating universal interfaces, such as the Model Context Protocol, and ensuring that agents can dynamically discover, register, and orchestrate tools without human intervention [47] [183]. Future research must prioritize the development of "Agent-First" APIs that are designed specifically for machine consumption rather than human readability, as well as the creation of synthetic ecosystems that allow agents to safely trial and error new tools in sandboxed environments [52] [168]. Additionally, addressing the "semantic-functional gap" requires new

training objectives that align semantic understanding with executable constraints, moving beyond simple next-token prediction to structured output verification [74] [46].

The Evolution from Static Prompts to Dynamic Orchestration

The trajectory of Large Language Model (LLM) agent development marks a definitive shift from rigid, prompt-dependent architectures toward fluid, dynamically orchestrated systems capable of navigating open-ended environments. Early iterations of agentic AI were largely constrained by static action spaces and in-context learning paradigms, where agents selected actions from a fixed, predefined set at every step. While effective in closed, narrowly scoped domains, this approach severely restricted planning capabilities in complex real-world scenarios and demanded impractical human effort to enumerate and implement all possible actions [62]. Furthermore, these monolithic designs often suffered from inefficiencies, as they indiscriminately incorporated all candidate tools into the agent's context, leading to increased token consumption and reasoning overhead without integrating tool registration into the reasoning procedure itself [115]. The limitations of such static frameworks are evident in their inability to adapt to unforeseen edge cases or recover from failures when predefined actions prove insufficient [62].

Contemporary research has responded to these constraints by advocating for dynamic architectures that decouple retrieval, planning, and execution, allowing agents to actively select, retrieve, and even generate tools based on specific task requirements. A central theme in this evolution is the move from fixed toolsets to modular systems where skills and tools are treated as composable, reusable assets. For instance, frameworks like SkillSmith address redundancy by compiling agent skills offline into minimal executable interfaces, enabling agents to access only relevant components at runtime and significantly reducing solve-stage token usage and thinking iterations [251]. Similarly, SkillX proposes a fully automated pipeline for constructing plug-and-play skill knowledge bases, distilling raw trajectories into hierarchical strategic plans and atomic skills that can be reused across different agents and environments, thereby mitigating the inefficiency of isolated learning [162]. This transition supports a more scalable infrastructure where agents do not merely recall static instructions but dynamically assemble capabilities, as seen in CUA-Skill, which encodes human computer-use knowledge into parameterized execution graphs to support dynamic skill retrieval and memory-aware failure recovery [277].

The sophistication of dynamic orchestration extends to the agent's ability to create new tools and adapt its own architecture in response to environmental feedback. Rather than relying solely on human-engineered APIs, modern agents can autonomously generate reusable components from UI interactions or synthesize new functions to fill capability gaps. The Deep Agent system exemplifies this through its Autonomous API & Tool Creation (AATC) module, which automatically generates reusable components, reducing operational costs and enhancing adaptability [205]. In the biomedical domain, the STELLA agent demonstrates a similar capacity for self-evolution by employing a "Tool Creation Agent" that autonomously discovers, tests, and integrates new bioinformatics tools into a dynamic "Tool Ocean," allowing the system to scale its expertise beyond predefined limits [STELLA: Self-Evolving LLM Agent for Biomedical Research]. This capability to induce programmatic skills on the fly allows agents to compose primitive actions into higher-level skills, improving efficiency and generalizability across diverse websites and tasks [274]. Such systems represent a fundamental departure from static prompting, moving toward agents that can rewrite their own operational boundaries and expand their action spaces through experience [153].

Moreover, the orchestration of these dynamic tools requires advanced retrieval and selection mechanisms to manage vast and heterogeneous tool ecosystems. As the number of available tools scales into the thousands, simple context injection becomes untenable due to token limits and semantic ambiguity. To address this, frameworks like TPTU-v2 introduce API retrievers that select pertinent APIs from extensive arrays, while Z-Space employs multi-agent collaborative architectures to filter and align user intents with specific tools without parameter tuning, drastically reducing token consumption [136; 122]. The importance of semantic context in this process cannot be overstated; theoretical and empirical analyses suggest that leveraging descriptive tool information is foundational for robust orchestration, enabling agents to adapt favorably in dynamic action spaces [183]. Additionally, the design of tool interfaces themselves is evolving to better support agent-native execution, with proposals like the Agent-First Tool API reconceiving interfaces as goal-achievement protocols that include built-in governance and error recovery, rather than simple CRUD endpoints [24].

Despite these advancements, challenges remain in balancing flexibility with efficiency and reliability. While dynamic architectures offer superior adaptability, they introduce complexities in coordination and cost. Research indicates that unrestricted flexibility, such as complete cyclic subtask graphs, can sometimes incur substantial inference costs and

coordination overhead compared to simpler forward execution models, particularly in domains with strict prerequisite chains [226]. Consequently, recent efforts focus on optimizing these workflows by identifying redundant execution patterns and transforming them into deterministic "meta-tools" that bundle multiple actions, thereby shortening execution paths and reducing failure rates [89]. Furthermore, the stability of tool-integrated agents remains a concern, as agents are vulnerable to errors at every stage of the invocation process, from reading documentation to processing responses, necessitating robust evaluation frameworks that account for environmental stochasticity and inter-tool dependencies [215; 262]. The field is thus converging on hybrid approaches that combine the adaptability of dynamic tool generation with the efficiency of compiled skill libraries and optimized orchestration protocols, paving the way for agents that are both autonomous and economically viable [50].

Core Challenges in Data Quality, Format Compliance, and Generalization

A primary bottleneck in the advancement of tool-augmented Large Language Models (LLMs) is the acute scarcity of high-quality, diverse training data capable of teaching models to adhere to strict API schemas. While early approaches relied on manual curation or limited synthetic generation, recent studies emphasize that the scale and fidelity of training corpora are directly correlated with model performance. [83] highlights this by constructing a comprehensive training set of 1,888 dialogues across 1,000 domains, noting that even with such data, significant gaps remain between open-source models and state-of-the-art closed systems. Similarly, [12] addresses the limitation of instruction tuning focusing only on basic language tasks by introducing ToolBench, a dataset covering over 16,000 real-world APIs, yet acknowledges that bridging the gap to models like ChatGPT requires not just volume but sophisticated reasoning strategies. The challenge of data quality is further compounded by the difficulty of ensuring accuracy in synthetic data; [191] argues that existing pipelines often lack coverage and precision, proposing a self-evolution synthesis process with dual-layer verification to generate accurate, complex tool-learning data. This sentiment is echoed by [219], which inverts the standard synthesis pipeline by exploring real APIs first and generating tasks backward from observed outcomes to guarantee label correctness, thereby mitigating the hallucination risks inherent in forward-generation methods. Furthermore, [160] addresses the limitations of real-world APIs, such as rate limits and access keys, by generating synthetic tool ecosystems that are both diverse and reliable for large-scale training, while [138] focuses on transforming existing task-adjacent datasets into API-based tasks to expand the training corpus without relying solely on synthetic generation.

Beyond data volume, a critical "format learning bottleneck" persists, where models struggle to generate syntactically valid function calls and adhere to specific parameter constraints. [8] introduces IFEval-FC to specifically test adherence to format instructions embedded in parameter descriptions, revealing that even advanced proprietary models frequently fail to follow basic formatting rules such as enclosing values in quotes or using ISO date formats. This inability to comply with strict schemas leads to execution failures, a problem [6] identifies as a tendency to hallucinate wrong API usage or generate inaccurate input arguments. To combat this, [27] proposes function masking techniques to minimize misleading naming conventions and enhance sensitivity to irrelevant functions, while [19] utilizes conditional masking to ensure outputs conform to desired formats for on-device models. Additionally, [133] provides a taxonomy of parameter failure, distinguishing between inherent model limitations causing name hallucination and input source issues leading to other failure patterns, suggesting that standardizing tool return formats is essential for reliability. The issue is further exacerbated in multilingual settings, as [108] warns that models often fail to generate language-invariant parameter values even when intent is understood, indicating that current benchmarks may not fully capture the nuances of global deployment.

Generalization to unseen tools and domains represents another formidable hurdle, as models often overfit to the specific APIs present in their training data. [12] demonstrates that while models can generalize to unseen APIs, performance often degrades without specialized retrieval mechanisms or depth-first search algorithms to expand the reasoning space. [235] attempts to address this by leveraging the semantic representation capabilities of frozen LLMs to handle massive pools of unseen tools without fine-tuning, though this approach still faces efficiency challenges compared to specialized training. The issue of scalability is further explored in [252], which argues that mapping each tool to a unique token creates a semantic bottleneck; instead, they propose encoding tools into hierarchical sequences to enable learning collaborative patterns from dense co-occurrences. Despite these advances, [14] notes that smaller language models, while efficient, struggle significantly with adhering to output formats and generalizing across domains without extensive fine-tuning, highlighting a trade-off between computational efficiency and robust tool-use capability. Moreover, [85] proposes a decision-aware framework to enhance generalizability over unseen tools by constructing samples with multiple decision branches, addressing the rigidity of template-driven approaches.

The complexity of multi-step and nested tool usage exacerbates these challenges, requiring models to maintain state and manage dependencies across multiple API calls. [63] and [34] both identify nested sequencing as a critical area where current models underperform, with [63] reporting that even GPT-4o achieves low accuracy on full sequence matches. [48] reinforces this by uncovering significant challenges in multi-hop scenarios, where leading models achieve less than 50% accuracy. The difficulty is not merely in selecting the right tool but in determining when a tool is necessary at all. [90] and [266] reveal a "knowing-doing gap," where models internally recognize the need for a tool based on hidden

states but fail to translate this recognition into action, often calling tools indiscriminately or not at all. This is compounded by the "tool usage inertia" observed in [16], where models rely on predictable sequential patterns rather than dynamic reasoning, leading to inefficiencies in novel contexts. Finally, the stability and realism of evaluation environments pose a significant barrier, with [295] proposing a framework where specialized LLMs simulate real API responses to act as "mirrors" to balance stability, scalability, and realness. Collectively, these studies illustrate that overcoming the bottlenecks in data quality, format compliance, and generalization requires a holistic approach involving verified data synthesis, strict format enforcement mechanisms, and evaluation frameworks that accurately reflect the complexity of real-world API interactions.

The Necessity of Comprehensive Benchmarking and Evaluation

The evolution of Large Language Models (LLMs) from static text generators to autonomous agents necessitates a fundamental shift in evaluation paradigms, moving beyond simple accuracy metrics to rigorous assessments of reliability, adaptability, and state management. Early benchmarks often operated under the simplifying assumption that tool usage is mandatory, failing to capture the nuanced reality where the necessity of external intervention is uncertain. This misalignment has led to the development of frameworks specifically designed to test an agent's ability to discern its own capability boundaries. Research indicates that current models frequently struggle to determine when not to use a tool, leading to unnecessary API calls that waste resources and degrade performance on general tasks [220]. This phenomenon is further elucidated by studies revealing a "knowing-doing gap," where models internally recognize the sufficiency of their own knowledge-evident in decodable hidden states-yet fail to suppress tool invocation during generation [90]. Crucially, this necessity is not a static property of a task but is model-adaptive; a problem solvable by a frontier model without assistance may require tools for a weaker variant, yet most benchmarks treat necessity as an absolute truth [266].

Beyond the binary decision of tool invocation, the reliability of agents collapses when faced with complex, multi-step workflows involving hidden dependencies. Traditional linear evaluations are insufficient for capturing the brittleness of state tracking in modern agents. Advanced frameworks that cast tool use as traversal over a dependency Directed Acyclic Graph (DAG) have revealed that performance declines sharply as dependency depth increases, with models often propagating stale argument values across steps [84]. This fragility is corroborated by query-driven benchmarks where even leading models achieve less than 50% accuracy on multi-hop scenarios, highlighting a severe deficit in maintaining context across sequential calls [48]. The challenge intensifies with nested tool calls, where the output of one function serves as the input for another; datasets constructed to test these nested sequences show that full sequence match accuracy remains critically low, often below 30% for state-of-the-art models [63] [34]. Furthermore, evaluations in dynamic, stochastic environments with interdependent tools reveal that top-tier models fail to exceed a 60% success rate, often succumbing to over-confidence or strategic defeatism when facing environmental noise [262].

The scalability of tool usage introduces further complications, particularly regarding long-context retention and the management of expansive tool repositories. As the catalog of available tools grows, models encounter a "semantic-functional gap," retrieving tools that are textually relevant but functionally inoperative due to parameter mismatches or authentication constraints [42]. This issue is compounded by the sheer volume of information; empirical studies document performance degradation of up to 91% as tool response lengths and conversation turns increase, indicating that long-context capabilities in natural language do not automatically translate to robust function calling [23]. In enterprise settings, where agents must coordinate cross-application workflows and adhere to strict policies, the failure rates are even more pronounced, with frontier models scoring below 10% on tasks requiring autonomous API discovery and policy adherence [273]. To address the lack of realistic dialogue dispersion, newer benchmarks emphasize multi-round, multi-party interactions, revealing that existing models struggle with the distribution of tool-related information across conversation turns [231]. Comprehensive stateful benchmarks demonstrate that complex tasks involving state dependency and canonicalization remain challenging even for the most capable proprietary models, exposing a significant gap between current capabilities and the requirements for reliable autonomous operation [100].

Future research must pivot towards developing evaluation frameworks that integrate dynamic environmental stochasticity, model-adaptive necessity metrics, and rigorous state-tracking tests. The community should move away from static, single-turn queries toward interactive sandboxes that simulate the "last mile" of commercial software automation, where tools are interdependent and prone to failure. There is a critical need for benchmarks that not only measure success rates but also diagnose the specific failure modes in the cognition-to-action transition, leveraging insights from hidden state probing to bridge the knowing-doing gap. Additionally, future evaluations must incorporate execution-grounded validation to distinguish between semantically plausible and functionally viable tool selections, ensuring that agents are robust against the semantic-functional gap inherent in large-scale API ecosystems. By adopting these comprehensive standards, the field can better gauge the readiness of LLM agents for real-world applicability and drive the development of architectures capable of managing complex, interdependent tool ecosystems without succumbing to context overload or hallucination.

Architectural Frameworks for Tool Integration and Management

The structural design governing how large language model (LLM) agents access, interpret, and manage external tools constitutes a primary determinant of system performance, scalability, and reliability. Current literature reveals a fundamental tension between the flexibility offered by dynamic tool registration and the computational efficiency of static, declarative descriptions. As agents are deployed in increasingly complex environments, the architectural challenge shifts from simple function calling to the orchestration of vast, heterogeneous toolsets while managing context window constraints and latency. A dominant theme in recent research is the optimization of tool representation to reduce token overhead without sacrificing semantic fidelity. Approaches such as [7] address the issue of diverse and redundant documentation by transforming lengthy tool descriptions into unified, concise instructions, significantly reducing token consumption while improving utilization. Similarly, [78] tackles semantic redundancy by automatically auditing and merging overlapping tools, subsequently filtering the catalog to fit within input limits. These methods contrast with more radical compression strategies like [103], which proposes a declarative YAML format to describe REST APIs, reducing the context cost of tool advertisement by orders of magnitude compared to traditional per-tool server processes. Furthermore, [252] addresses the scalability crisis of mapping tools to unique tokens by encoding tools into hierarchical sequences, enabling the model to learn collaborative patterns from dense co-occurrences rather than sparse tool IDs.

Beyond compression, the mechanism of tool retrieval and selection has evolved from static lists to dynamic, graph-based, and execution-validated architectures. Traditional retrieval often suffers from a "semantic-functional gap," where tools appear relevant textually but fail during execution due to parameter mismatches or authentication issues. [42] addresses this by implementing a plan-execute-evaluate cycle that validates tools empirically rather than relying solely on semantic similarity. To handle the scale of enterprise environments, researchers are increasingly leveraging knowledge graphs and unified vector spaces. [71] and [107] both propose embedding tools and their parent agents in a shared space, linked via metadata or graph edges. This allows for granular retrieval that preserves the context of agent bundles while avoiding the dilution caused by coarse summaries. Furthermore, [ScaleMCP] introduces dynamic synchronization with Model Context Protocol (MCP) servers, enabling agents to autonomously update their tool memory through CRUD operations, thereby mitigating the inconsistencies of monolithic local repositories. Complementing these structural approaches, [35] argues that semantic relevance is insufficient, proposing a training-free method that exposes only the minimal next-step tool frontier based on causal sufficiency to reduce premature actions.

The efficiency of tool invocation is further optimized through compiler-based approaches and speculative execution. [251] proposes compiling skill packages offline into minimal executable interfaces, drastically reducing redundant reasoning and context injection at runtime. Complementing this, [165] and [70] introduce speculative execution mechanisms that hide tool latency by predicting and executing likely tool calls in parallel with the LLM's reasoning process. These architectural innovations aim to break the serial "LLM-tool" loop that traditionally bottlenecks agent throughput. Additionally, [16] exploits "tool usage inertia" by constructing directed graphs from historical trajectories to predict sequential tool patterns, reducing the need for repeated LLM inference during selection.

However, the move toward dynamic and automated tool management introduces new vulnerabilities. Studies such as [215] and [82] highlight that agents are highly susceptible to errors and adversarial manipulation of tool descriptions, where minor edits can bias selection toward malicious or suboptimal tools. This necessitates robust security frameworks like [97], which enforces attribute-based access control to mitigate cross-tool security risks. The evolution of tool integration also extends to the automatic generation and adaptation of tools themselves. Frameworks like [111] and [TOOLMAKER] demonstrate the ability to synthesize executable tools directly from unstructured API documentation or scientific code repositories. [62] pushes this further by enabling agents to dynamically compose actions using general-purpose programming languages, moving beyond fixed action sets. Finally, the field is transitioning from single-call correctness to the robustness of long-horizon multi-tool chains, requiring architectures that can handle state consistency and complex dependency modeling, as emphasized in [50]. The integration of middleware layers, as suggested in [267], further shields agents from environmental complexity, allowing them to operate effectively in massive knowledge bases and databases.

Declarative Descriptions and Enterprise-Scale Tool Libraries

The transition from experimental agent prototypes to enterprise-grade deployments necessitates a fundamental architectural shift in how tools are registered, described, and executed. In large-scale organizational settings, the prevailing Model Context Protocol (MCP) standard, while effective for unifying interfaces, encounters significant scalability bottlenecks when applied naively. The primary structural deficiency lies in the per-tool server process model, where every API integration requires a dedicated deployment with its own dependency tree and credential handling. Empirical analysis reveals that the vast majority of these servers function merely as thin wrappers around existing REST APIs, creating unnecessary operational overhead [103]. Furthermore, the standard registration model forces context window consumption to grow linearly with the number of available tools. This linear growth compels real-world deployments to expose only a fractional subset of an organization's actual API catalog, severely limiting agent autonomy and utility [103]. Studies on long-context models indicate that performance degrades significantly-by up to 85%-as the number of tools in the context increases, a phenomenon known as the "lost in the middle" effect, which renders monolithic tool lists impractical for complex enterprise environments [23].

To mitigate these inefficiencies, declarative description languages have emerged as a critical solution for decoupling tool definition from execution logic. By utilizing formats such as YAML to describe endpoints, authentication mechanisms, pagination strategies, and response shaping in a single file, organizations can eliminate the need for per-API server processes. Instead, a shared runtime interprets these declarative files, centralizing security boundaries and authorization decisions while exposing tools through a fixed-size interface independent of the catalog's magnitude [103]. This approach has demonstrated the ability to reduce the token cost of tool advertisement by orders of magnitude, compressing schema requirements from approximately 142,000 tokens to roughly 1,000 tokens without sacrificing functional coverage [103]. Complementing this, frameworks like ToolRegistry offer protocol-agnostic management that reduces integration code by 60% and supports concurrent execution, further alleviating the computational burden of managing diverse toolsets [170]. Similarly, Z-Space employs a multi-agent collaborative architecture to filter thousands of heterogeneous tools, achieving substantial reductions in token consumption while maintaining high invocation accuracy in production environments [122].

Beyond mere token reduction, declarative approaches facilitate the creation of versioned, auditable Enterprise Tool Libraries that support dynamic retrieval and synchronization. Systems like ScaleMCP address the limitations of static, monolithic repositories by introducing auto-synchronizing storage pipelines that treat MCP servers as the single source of truth, allowing agents to dynamically add tools to their memory via CRUD operations [47]. This dynamic capability is complemented by advanced retrieval strategies that operate on unified vector spaces embedding both tools and their parent agents. Tool-to-Agent Retrieval, for instance, links fine-grained tool capabilities with higher-level agent contexts, preventing the context dilution that occurs when coarse agent descriptions obscure specific functionalities [71]. Other approaches, such as ToolScope, utilize automatic merging and context-aware filtering to resolve semantic redundancies in large toolsets, ensuring that only the most relevant tools are passed to the model [78].

However, the reliance on textual descriptions for tool selection introduces vulnerabilities that declarative systems must address. Research indicates that LLM tool selection is surprisingly fragile and susceptible to manipulation; minor edits to tool names or descriptions can bias agents toward specific tools, increasing their usage by over ten times [194]. This vulnerability extends to security risks such as tool poisoning and prompt injection attacks, where malicious actors can inject deceptive tool documents to hijack agent workflows [210] [145]. Consequently, the design of tool interfaces requires rigorous "docstring engineering" to optimize interaction reliability and mitigate ambiguity-induced failures [52]. Emerging paradigms like Agent-First Tool APIs propose reconceiving interfaces as goal-achievement protocols rather than simple form-submission endpoints, incorporating confidence scores and structured error recovery to enhance robustness [24].

The evolution toward declarative and scalable tool management also intersects with the need for standardized evaluation in complex, interdependent environments. Benchmarks such as ComplexMCP highlight that current agents struggle with stateful, stochastic environments where tools exhibit strong interdependencies, often failing to exceed 60% success rates in realistic scenarios [262]. To bridge the gap between isolated API calls and complex orchestration, frameworks are increasingly adopting graph-based planning and causal filtering to manage tool dependencies and reduce premature actions [53] [35]. Ultimately, the synthesis of declarative descriptions, centralized runtimes, and

robust retrieval mechanisms forms the backbone of sustainable enterprise tool integration, enabling organizations to scale from dozens to thousands of tools while maintaining security, efficiency, and agent reliability.

Knowledge Graph-Based Retrieval and Agent Routing

As tool repositories within Large Language Model (LLM) multi-agent systems expand to encompass thousands of Model Context Protocol (MCP) servers and functions, traditional retrieval mechanisms based solely on vector similarity or lexical matching have proven insufficient for identifying optimal agent-tool pairs. The fundamental limitation of existing approaches lies in their granularity: agent-only retrievers often match queries against coarse agent descriptions, thereby obscuring fine-grained tool capabilities, while tool-only retrieval treats functions in isolation, ignoring the complementary benefits of coherent agent bundles required for multi-step tasks [107]. This dichotomy creates a retrieval gap where neither the holistic context of an agent nor the specific utility of a tool is fully leveraged, leading to suboptimal routing decisions in complex workflows [71]. To address these architectural deficiencies, recent research has pivoted toward graph-based structures that explicitly model the relationships between tools and their parent agents, enabling joint retrieval and sophisticated traversal strategies that outperform isolated selection methods.

The primary innovation in this domain is the "Agent-as-a-Graph" framework, which represents both tools and their parent agents as nodes within a unified knowledge graph, connected by edges denoting ownership or functional dependency. Unlike prior methods that concatenate agent and tool descriptions into a single vector space—a practice that suffers from multi-hop query limitations and context dilution—this approach maintains distinct semantic representations while enabling structural navigation [107]. The retrieval process involves an initial vector search to identify relevant nodes of both types, followed by a type-specific weighted reciprocal rank fusion (wRRF) mechanism. This fusion strategy applies distinct weighting parameters to agent and tool nodes, allowing the system to balance tool-specific relevance with agent-level capability before traversing the graph to aggregate the final set of parent agents. Empirical evaluations on the LiveMCPBenchmark demonstrate that this graph-augmented approach significantly outperforms state-of-the-art baselines, achieving substantial improvements in Recall@5 and nDCG@5 metrics compared to retrievers that treat agents and tools in isolation [107].

Complementing this structural approach, the "Tool-to-Agent Retrieval" framework further refines the concept of shared vector spaces by explicitly linking tools to agents through metadata relationships without collapsing them into a single coarse description. This method addresses the scalability challenge where forwarding all tool descriptions to a model is computationally prohibitive, yet relying solely on agent summaries risks missing critical functional nuances. By embedding tools and agents in a unified space and utilizing a bipartite graph structure for traversal, the system can dynamically decide whether to return an individual tool for specific queries or an entire agent bundle for broader capabilities [71]. This granular routing mechanism mitigates the context dilution inherent in chunking many tools together and supports both direct and step-wise querying paradigms, ensuring robust performance across diverse embedding models and multi-step workflows [71].

Beyond static graph construction, dynamic evolution and adaptive structuring of tool repositories offer additional layers of optimization. Frameworks such as GATE (Graph-based Adaptive Tool Evolution) extend these concepts by dynamically constructing and evolving hierarchical graphs of reusable tools across multiple scenarios, balancing tool quantity and complexity to enhance efficiency in open-ended tasks [203]. Similarly, AutoTool exploits the phenomenon of "tool usage inertia," constructing directed graphs from historical agent trajectories to model predictable sequential patterns, thereby reducing the reliance on repeated LLM inference for tool selection [16]. These graph-based methodologies contrast sharply with purely semantic retrieval systems that often fail to capture functional viability or execution constraints, a limitation highlighted by frameworks like GRETEL which emphasize the "semantic-functional gap" where textually relevant tools may be functionally inoperative [42].

The integration of knowledge graphs also facilitates more sophisticated security and governance mechanisms within agent routing. As agents navigate complex toolchains, the risk of cross-tool vulnerabilities and data pollution increases. Graph-based tracking allows for the identification of source-to-sink dataflows that only emerge through tool composition, enabling systems like ChainFuzzer to detect multi-tool vulnerabilities that single-hop testing misses [60]. Furthermore, structured representations support attribute-based access control frameworks like AgentGuard, which rely on clear definitions of tool relationships to enforce security policies across single-tool and cross-tool execution contexts [97]. By moving beyond flat lists of tools to interconnected knowledge graphs, architectural frameworks for tool integration can achieve higher accuracy, better scalability, and improved robustness against both functional failures and security threats, marking a significant shift from simple retrieval to intelligent, context-aware orchestration [50].

Compiler-Based Optimization and Boundary Guidance

The traditional paradigm of integrating large language models (LLMs) into autonomous agents has relied heavily on injecting full skill contexts or extensive tool documentation directly into the reasoning loop. While this approach provides flexibility, it introduces significant inefficiencies, including irrelevant context injection, repeated skill-specific reasoning, and excessive token consumption. To address these bottlenecks, recent architectural frameworks have shifted toward a "boundary-first" compiler-runtime approach. This methodology compiles skills offline into minimal executable interfaces, allowing agents to access only relevant components dynamically during the solve stage. A primary example of this shift is the proposal to compile skill packages offline to extract fine-grained operational boundaries, enabling agents to execute only necessary components at runtime [251]. By transforming passive textual guidance into active, executable guardrails, this approach reduces solve-stage token usage by over 57% and thinking iterations by nearly 43% compared to raw skill injection. This contrasts sharply with earlier methods that automate multi-step reasoning but still rely on selecting demonstrations from a task library that may retain redundant contextual overhead [249].

The core innovation in compiler-based optimization lies in upgrading skills from advisory text into executable Program Functions (PFs) that act as guardrails. These functions activate only on failure-prone states to modify actions or inject corrective context, moving beyond the advisory nature of traditional textual skills [209]. This transformation aligns with the broader trend of treating tools not merely as external APIs but as structured, composable units of logic. For instance, specific frameworks encode human computer-use knowledge into skills coupled with parameterized execution and composition graphs, creating a scalable infrastructure for reliable agent development [277]. Similarly, other approaches bridge the grounding gap in web agents by pairing parameterized action programs with step-level natural language guidance, enabling both direct execution and agent-driven adaptation [225]. These methods collectively suggest that the future of agent architecture involves typed, efficient tool interfaces rather than stateless prompt-driven interactions.

Efficiency gains in these frameworks are often achieved by minimizing the context window required for tool registration and execution. Recent algorithms address the inefficiency of indiscriminately incorporating all candidate tools into the agent's context by allowing LLMs to selectively register tools as needed, integrating the registration process directly into the reasoning procedure [115]. This selective registration reduces computational costs by over 50% in multi-step tasks. Furthermore, the concept of "meta-tools" takes this optimization further by bundling recurring sequences of tool calls into deterministic, composite tools, thereby bypassing unnecessary intermediate LLM reasoning steps [89]. Such compilation strategies effectively reduce the latency and resource consumption associated with sequential tool invocation, a problem also addressed by speculative execution methods that hide tool latency by predicting recurring tool-call sequences [165].

Despite the advantages of compiled interfaces, challenges remain regarding the granularity of boundaries and the adaptability of compiled skills to novel environments. Automated pipelines have been proposed to construct hierarchical skill knowledge bases through iterative refinement, yet they highlight the difficulty of generalizing skills across different agents and environments without redundant exploration [162]. The tension between rigid, compiled efficiency and flexible, dynamic adaptation is evident when comparing boundary-guided systems with those that rely on dynamic action creation. Some frameworks argue against fixed action sets, proposing instead that agents dynamically create and compose actions as programs to handle open-ended scenarios [62]. However, compiler-based approaches counter that without the structure of pre-compiled boundaries, agents suffer from trajectory instability and locally myopic generation, issues that recent meta-adaptive frameworks attempt to mitigate through lookahead strategies and trajectory convergence mechanisms [289].

The design of the interface itself plays a critical role in the success of compiler-based optimization. Proper structure, such as State-Machine Augmented Generation, can close reasoning gaps between base models by representing business logic as state machines used as tools [217]. This structural rigor is echoed in design paradigms that reconceive interfaces as goal-achievement protocols rather than simple form-submission endpoints, incorporating multi-phase reasoning loops directly into the tool contract [24]. Moreover, the quality of the compiled artifact depends heavily on the clarity of the source documentation; transforming diverse and lengthy tool documentation into unified, concise instructions significantly improves agent performance and reduces token consumption [7]. Ultimately, the transition from context-heavy injection to boundary-guided compilation represents a maturation of agent architecture, supporting

the deployment of production-grade agents that require verifiable planning and scalable coordination [47]. By compiling skills into minimal, executable boundaries, these frameworks not only reduce operational costs but also enhance the reliability and interpretability of agent behaviors [251; 209].

Hierarchical and Self-Reflective Agent Designs

The transition from single-tool invocation to complex multi-tool orchestration necessitates a fundamental architectural shift from flat selection mechanisms to hierarchical and self-reflective designs. As the scope of available tools expands into the thousands, the cognitive load on Large Language Models (LLMs) to simultaneously reason about task decomposition, tool selection, and parameter instantiation becomes unsustainable. Contemporary frameworks address this combinatorial explosion by decomposing the agent's workflow into distinct, specialized stages: retrieval, solving, and reflection. This decomposition not only manages context window constraints but also isolates failure modes, allowing for targeted recovery strategies that flat architectures cannot support [50]. A paradigmatic example of this approach is AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls, which operationalizes this tripartite structure by employing a hierarchical retriever to filter candidates from a pool of over 16,000 APIs before engaging a solver, supplemented by a self-reflection mechanism that re-activates the entire pipeline if the initial solution proves impracticable. This design directly counters the limitations of single-pass selection, where models frequently conflate semantic relevance with functional viability.

The retrieval stage within these hierarchical systems faces the dual challenges of semantic ambiguity and context saturation. Traditional approaches that inject full tool descriptions into the prompt fail as libraries scale, prompting the development of declarative and compressed interface languages. DADL: A Declarative Description Language for Enterprise Tool Libraries in LLM Agent Systems introduces a YAML-based format that decouples tool definition from server deployment, reducing the context cost of tool advertisement by orders of magnitude while maintaining centralized authorization. Similarly, EASYTOOL: Enhancing LLM-based Agents with Concise Tool Instruction purifies essential information from diverse documentation into unified instructions, significantly improving selection accuracy. However, semantic similarity alone often fails to capture functional viability, creating a "semantic-functional gap" where retrieved tools are textually relevant but executionally broken due to parameter mismatches or authentication failures. GRETEL: A GOAL-DRIVEN RETRIEVAL AND EXECUTION-BASED TRIAL FRAMEWORK FOR LLM TOOL SELECTION ENHANCING addresses this by subjecting semantically retrieved candidates to sandboxed plan-execute-evaluate cycles, using empirical execution feedback to re-rank tools based on demonstrated utility rather than textual similarity. This empirical validation is critical for bridging the gap between coarse agent descriptions and fine-grained tool capabilities, a challenge further addressed by Tool-to-Agent Retrieval: Bridging Tools and Agents for Scalable LLM Multi-Agent Systems, which embeds both tools and parent agents in a shared vector space to enable granular retrieval without context dilution.

Beyond retrieval, hierarchical designs extend to the execution phase through graph-based structures and predictive modeling to reduce inference overhead. AutoTool: Efficient Tool Selection for Large Language Model Agents exploits "tool usage inertia"-the predictable sequential patterns in tool invocation-to construct directed graphs that bypass repeated LLM inference for tool selection. This statistical approach complements logical reasoning frameworks like Reverse Chain: A Generic-Rule for LLMs to Master Multi-API Planning, which utilizes backward reasoning to determine the optimal sequence for API selection and argument completion. These methods collectively suggest that efficient orchestration requires moving beyond reactive prompting toward proactive, structure-aware planning. Furthermore, the rigidity of predefined toolsets in hierarchical systems is increasingly being challenged by dynamic action generation. DynaSaur 1 : Large Language Agents Beyond Predefined Actions proposes a framework where agents dynamically create and compose actions as programs when predefined tools are insufficient, accumulating these generated actions for future reuse. This capability transforms the hierarchy from a static filter into an adaptive constructor, allowing agents to navigate open-ended environments where the necessary tools do not yet exist.

Self-reflection serves as the critical corrective layer that stabilizes these complex hierarchies against error propagation. In black-box settings where API implementations are hidden, WISSNYF: TOOL GROUNDED LLM AGENTS FOR BLACK BOX SETTING leverages program synthesis to strategize tool usage, verifying solutions prior to implementation to ensure safety. The necessity for such robust reflection is underscored by findings in More Vulnerable than You Think: On the Stability of Tool-Integrated LLM Agents, which reveals that open-source models are particularly susceptible to errors at every stage of the tool invocation process, from documentation reading to response processing. Consequently, modern architectures are evolving to embed verification directly into the interface design. Agent-First Tool APIs: Rethinking Enterprise Service Interfaces for LLM-Native Execution reconceives interfaces as

goal-achievement protocols with built-in recovery phases, structuring invocations as multi-phase reasoning loops that include verification and error recovery steps. This shift is essential for enterprise adoption, where benchmarks like ComplexMCP: Evaluation of LLM Agents in Dynamic, Interdependent, and Large-Scale Tool Sandbox reveal that even top-tier models struggle with strategic defeatism and over-confidence in interdependent environments. Ultimately, the integration of hierarchical retrieval, dynamic action creation, and rigorous self-reflective loops represents the frontier of scalable agent design, enabling systems to navigate the complexities of real-world API ecosystems with greater autonomy and resilience.

Data Synthesis and Training Methodologies for Tool Use

The efficacy of Large Language Model (LLM) agents in tool-use scenarios is fundamentally contingent upon the quality, diversity, and scalability of training data. A primary bottleneck in this domain is the scarcity of high-quality, verifiable trajectories, particularly for complex, multi-step tasks involving real-world APIs. To address data limitations, recent research has pivoted toward sophisticated synthetic data generation frameworks that prioritize structural diversity and semantic coherence over simple volume. [160] introduces a scalable ecosystem for generating synthetic tools, comprising automated tool generation, realistic behavior simulation, and rigorous auditing to ensure consistency, thereby overcoming the instability and access restrictions of real-world APIs. Similarly, [191] employs a self-evolution synthesis process involving multiple agents and a complexity evaluator to curate a comprehensive API pool, utilizing a dual-layer verification system to guarantee data accuracy. Expanding on the need for linguistic variety, [155] argues that prior methods overlook the diversity of user requests and argument coverage, proposing a metric-optimized generation approach that significantly improves out-of-distribution performance. Furthermore, [218] addresses the lack of coherence in synthesized dialogues by implementing graph-based sampling for relevant tool combinations and a planned-generation strategy to create natural, multi-turn interactions. These approaches collectively shift the paradigm from static, rule-based data creation to dynamic, agent-driven synthesis that mirrors real-world complexity, while [219] offers a complementary approach by inverting the synthesis pipeline to generate tasks backward from observed outcomes on real MCP servers, guaranteeing label correctness by construction.

Beyond data generation, the methodology of training agents has evolved from static trajectory imitation to dynamic, environment-interaction-based learning. Traditional supervised fine-tuning (SFT) often leads to models that memorize specific solution paths, failing to generalize to novel tools or evolving environments. [286] proposes ToolMaster, a framework that replaces the imitation of golden trajectories with a trial-and-execution paradigm. By training agents to actively explore environments and learn from self-correction via reinforcement learning, ToolMaster fosters robustness against unseen tools. This aligns with findings in [201], which demonstrates that agents relying on high-failure trial-and-error are inefficient; instead, it advocates for a GUI-based depth-first search to systematically explore unit functions and construct a curriculum of composite skills. Similarly, [268] enables agents to autonomously master unfamiliar software through experiential learning, utilizing a World State Model for trajectory assessment and Group Relative Policy Optimization (GRPO) to refine policies based on success and failure. These works underscore a critical consensus: robust generalization requires agents to learn through interaction and adaptation rather than mere replication of expert demonstrations.

Reinforcement learning (RL) has emerged as a pivotal mechanism for optimizing tool-use policies, particularly in decoupling planning capabilities from response generation. [178] introduces RL with Tool-use Rewards (RLTR), which isolates the planning module for single-objective optimization using a reward signal based on tool-use completeness, effectively bypassing the need for verifiable final answers. This focus on process over outcome is further refined in [139], which assigns rewards at each interaction step to address the sparse reward problem inherent in long-horizon tasks. Complementing this, [157] identifies entropy reduction as a supervisory signal, demonstrating that dense process rewards guided by entropy metrics can significantly enhance performance while reducing unnecessary tool calls. However, challenges remain in balancing exploration with efficiency. [281] addresses this by integrating strategic entropy-based exploration into GRPO to improve reasoning transparency and parameter extraction, while [221] offers a modular framework supporting various RL algorithms and automated reward generation to lower the barrier for domain-specific adaptation.

The structural representation of skills and tools also plays a crucial role in training efficiency and generalization. Rather than treating tools as static API definitions, several frameworks advocate for hierarchical skill abstraction. [162] proposes distilling raw trajectories into a three-tiered hierarchy of strategic, functional, and atomic skills, enabling reusable knowledge bases that improve transferability across agents. In the realm of computer use, [277] encodes human interaction knowledge into parameterized execution graphs, allowing for dynamic skill retrieval and failure recovery. Conversely, some approaches suggest modifying the tool interface itself to better suit agent learning. [18] presents a curriculum learning framework that optimizes tool interfaces without execution traces, while [204] represents tools as learnable embeddings ("toolkens"), allowing frozen models to access massive toolsets without context window

limitations. These diverse strategies highlight a growing recognition that effective tool learning requires not just better data or algorithms, but also optimized representations of the tools and skills themselves.

Despite these advancements, tensions exist between the flexibility of dynamic learning and the efficiency of static optimization. While [62] argues for dynamically creating actions via program synthesis to handle open-ended scenarios, [16] observes "tool usage inertia," suggesting that statistical modeling of sequential patterns can reduce inference costs by up to 30% compared to repeated LLM-based selection. Furthermore, [22] proposes a novel paradigm where functions themselves are treated as learnable parameters, updated by the LLM without modifying the model weights, offering a distinct alternative to traditional fine-tuning. The integration of these methodologies suggests a future where agents combine synthetic data scalability, interactive reinforcement learning, and structured skill representations to achieve human-level proficiency in tool use. Future research must address the "knowing-doing" gap by developing curricula that explicitly train agents to recover from failures, as seen in [171], and by creating standardized benchmarks that evaluate not just task success but the efficiency and robustness of the learning process itself. Additionally, the development of "agent-first" APIs that are designed specifically for machine consumption, rather than adapted from human interfaces, could further bridge the semantic-functional gap identified in current literature.

Synthetic Data Generation via Multi-Agent Frameworks

The scarcity of high-quality, diverse, and verifiable training data remains a primary bottleneck in developing robust function-calling capabilities for Large Language Models (LLMs). While real-world API interactions provide ground truth, they are often difficult to annotate at scale, suffer from privacy constraints, or lack the necessary diversity to ensure model generalization. Consequently, recent research has pivoted toward automated synthetic data generation pipelines that leverage multi-agent frameworks to construct complex, realistic interaction trajectories. These systems move beyond simple single-turn prompt-response pairs, aiming to simulate the nuanced, multi-step reasoning required in real-world agentic deployments.

A dominant strategy in this domain involves the use of multi-agent architectures to generate dialogues that span diverse scenarios while maintaining strict quality control. [280] introduces an automated pipeline that utilizes a multi-agent framework to support dialogue generation, ensuring both diversity and accuracy through a multi-stage evaluation system. By starting from reliable tools in public benchmarks, this approach mitigates the issue of unreliable APIs often found in synthetic datasets. Similarly, [191] employs a self-evolution synthesis process where multiple agents interact under the guidance of a complexity evaluator to curate a comprehensive API pool. This method incorporates a dual-layer verification system combining rule-based and model-based checks to ensure data accuracy, demonstrating that models trained on such synthesized data can achieve performance comparable to proprietary giants like GPT-4. These approaches highlight a consensus that multi-agent interaction, rather than single-model generation, is critical for scaling data quality without sacrificing diversity.

Addressing the specific challenge of compositional and multi-turn interactions, several frameworks have adopted bottom-up and top-down construction methods to fill gaps in interaction patterns. [61] presents the BUTTON approach, which generates synthetic compositional instruction tuning data by first constructing simple atomic tasks and then building complex compositional tasks using heuristic strategies. This is followed by a top-down phase within a multi-agent environment where simulated humans, assistants, and tools interact to gather multi-turn trajectories. This ensures that the generated data captures the necessary task compositionality often missing in single-turn datasets. Complementing this, [218] addresses the lack of coherence in existing synthetic data by introducing a Graph-based Sampling strategy for relevant tool combinations and a Planned-generation strategy to guide coherent dialogue synthesis. By enabling multiple agents to synthesize dialogue data interactively, TOOLFLOW significantly improves the naturalness of the resulting conversations, addressing the disconnect between random tool sampling and realistic user intent.

The fidelity of synthetic data is further enhanced by grounding generation in real API behaviors and verified outcomes, moving away from purely hallucinated environments. [219] inverts the standard synthesis pipeline by first allowing a strong LLM to explore real APIs along graph-guided DAG structures and then synthesizing tasks backward from observed outcomes. This guarantees label correctness by construction and handles environment drift through a retrieval-augmented simulator. In enterprise contexts, where specific business scenarios dictate tool usage, [132] demonstrates a tailored pipeline that synthesizes and augments scenario-specific function-calling data, proving that domain-adapted synthetic data can surpass general-purpose models in accuracy. Furthermore, [160] tackles the limitations of real-world API availability and stability by generating entirely synthetic tool ecosystems. This framework includes components for tool generation, simulation, and auditing, enabling scalable training in controllable environments that span twice as many domains as prior work.

Despite these advancements, contradictions and gaps remain regarding the evaluation of synthetic data quality and its impact on model robustness. While many pipelines claim high accuracy, [231] reveals that existing benchmarks often overlook the dispersion of tool-related information throughout a dialogue, leading to low scores on realistic multi-round scenarios. This suggests that while multi-agent generation improves diversity, the resulting data may still fail to capture the intricate dependencies of long-horizon tasks. Additionally, [141] highlights that current evaluation methodologies often miss critical weaknesses in model resilience to input perturbations and toolkit expansions, indicating that synthetic data generation must evolve to include adversarial scenarios. The tension between data diversity and linguistic variety is also noted in [155], which argues that prior work focuses too heavily on function invocation patterns while neglecting the linguistic diversity of user queries and argument coverage, a gap that limits out-of-distribution performance.

The integration of these synthetic data strategies into broader training methodologies underscores a shift toward more sophisticated, agent-centric learning paradigms. As noted in [36], the workflows for tool use are becoming increasingly complex, requiring unified taxonomies to compare different generation and training frameworks effectively. Moreover, the efficiency of utilizing this data is paramount; [11] posits that without explicit orchestration policies, agents trained on rich synthetic data may still suffer from excessive tool calls and redundant reasoning steps. Therefore, the future of synthetic data generation lies not only in creating larger datasets but in producing data that explicitly teaches models to balance answer quality with execution cost, a capability that current multi-agent frameworks are only beginning to address through mechanisms like progress awareness and utility-guided decision making.

Reinforcement Learning for Structured Reasoning and Policy Alignment

The transition from static text generation to dynamic decision-making in Large Language Models (LLMs) necessitates a fundamental shift in training methodologies, moving beyond Supervised Fine-Tuning (SFT) which often instills superficial pattern matching rather than robust reasoning strategies [281]. While SFT effectively internalizes syntax, it fails to address the sparse reward signals inherent in complex, multi-step tool usage, where a single parameter error can invalidate an entire trajectory [99]. Consequently, recent research has pivoted toward Reinforcement Learning (RL) frameworks that explicitly align the reasoning process with tool-call decisions, treating function calling as a sequential decision-making problem rather than a monolithic generation task. A primary innovation in this domain is the development of composite reward structures that decouple format correctness from reasoning quality. The R2IF framework, for instance, addresses the misalignment between Chain-of-Thought (CoT) generation and final action by integrating a Chain-of-Thought Effectiveness Reward (CER) and a Specification-Modification Value (SMV) alongside standard format constraints [265]. This approach ensures that the model's internal logic is not only correct but also interpretable and adherent to specification constraints, significantly enhancing reliability in tool-augmented deployments.

Complementing composite rewards, step-grained optimization strategies have emerged to refine policy learning at the granularity of individual tool interactions. The StepTool framework reframes tool learning by assigning rewards at each interaction based on invocation success and contribution to the overall task, thereby mitigating the credit assignment problems prevalent in end-to-end optimization [139]. Similarly, the RLTR framework decouples action planning from answer summarization, utilizing tool-use completeness as a direct reward signal to optimize the planning module independently of the final response quality [178]. This decoupling is critical for addressing the imbalanced objective allocation often seen in multi-objective optimization, allowing agents to develop robust planning capabilities even in the absence of verifiable final answers. To further enhance the precision of these rewards, process reward models like ToolPRM have been introduced to score internal steps of a function call, revealing a key principle for structured output scaling: "explore more but retain less" due to the unrecoverable nature of errors in structured generation [46].

The challenge of exploration in the vast action space of function calling has led to novel entropy-based strategies. Standard Group Relative Policy Optimization (GRPO) often suffers from insufficient exploration, leading to suboptimal policy convergence. To counter this, the FunRL framework introduces entropy-enhanced advantage estimation, which strategically incorporates CoT entropy into the reward calculation to promote diverse reasoning paths while maintaining optimization stability [281]. This method leverages the structured language generation capabilities of code-pretrained models, demonstrating that encouraging diverse thought processes yields superior performance on complex multi-function scenarios. Furthermore, meta-adaptive exploration frameworks like MAXS employ lookahead strategies and trajectory convergence mechanisms to balance global effectiveness with computational efficiency, selecting stable reasoning steps based on inter-step trend slopes [289]. These approaches collectively address the exploration-exploitation dilemma by ensuring that agents do not prematurely converge on locally optimal but globally suboptimal tool usage patterns.

A critical tension exists in the literature regarding the optimal length and structure of reasoning trajectories. While many frameworks advocate for extensive CoT to improve interpretability, empirical studies reveal a non-monotonic relationship between reasoning budget and accuracy. The study "Brief Is Better: Non-Monotonic Chain-of-Thought Budget Effects in Function-Calling Language Agents" demonstrates that brief reasoning (approximately 32 tokens) significantly improves accuracy by acting as a function-routing step, whereas extended reasoning can degrade performance by introducing hallucinations and wrong function selections [73]. This finding challenges the assumption that longer reasoning is inherently superior and suggests that structured, concise reasoning templates may be more effective for parameter accuracy. Addressing this trade-off, the FunReason framework introduces a Self-Refinement Multiscale Loss (SRML) that dynamically balances the learning objectives of the reasoning process and the function call result, preventing the model from over-optimizing for lengthy reasoning at the expense of parameter precision [FunReason: Enhancing Large Language Models' Function Calling via Self-Refinement Multiscale Loss and Automated Data Refinement].

Finally, the alignment of policy with user intent in multi-turn scenarios requires explicit modeling of progress awareness. The PARL-MT framework integrates progress awareness directly into RL training to reduce contextual

redundancy and improve the alignment between local actions and global task completion, addressing the limitations of reducing multi-turn training to isolated single-turn samples [20]. This is supported by data synthesis pipelines like ToolACE, which utilize self-evolution processes to generate high-quality, diverse training data covering complex API interactions, enabling smaller models to achieve performance comparable to larger proprietary systems [191]. Together, these advancements signify a paradigm shift where the quality of the reasoning trajectory is treated as a first-class citizen in the optimization objective, ensuring that autonomous agents are not only accurate in their final outputs but also robust, interpretable, and efficient in their decision-making processes.

Conclusion

The literature on tool learning and function calling reveals a definitive paradigm shift from static, prompt-dependent interactions toward dynamic, self-evolving agentic systems. The field has progressed beyond simple retrieval-augmented generation to sophisticated architectures that treat tool use as a sequential decision-making problem, necessitating a move from supervised fine-tuning to reward-driven policy learning. While early approaches relied on frozen models and in-context learning, current research emphasizes the critical role of reinforcement learning in optimizing long-horizon planning, addressing sparse reward signals, and bridging the gap between linguistic understanding and real-world execution. This evolution underscores a consensus that robust agency requires not just the ability to call functions, but the capacity to adapt strategies, recover from failures, and autonomously expand action spaces through experience.

A central tension identified across diverse studies lies in balancing flexibility with efficiency and reliability. The transition from rigid, predefined action sets to fluid, dynamically orchestrated systems allows agents to navigate open-ended environments and compose novel tools; however, this flexibility introduces significant challenges regarding computational cost, latency, and the "semantic-functional gap" where textually relevant tools fail during execution. To mitigate these issues, the community is converging on hybrid solutions that combine compiler-based optimizations, speculative execution, and hierarchical skill abstraction. By compiling skills into minimal executable interfaces and leveraging graph-based retrieval mechanisms, researchers are developing agents that can manage vast tool ecosystems without succumbing to context overload or inference bottlenecks. Furthermore, the emergence of declarative description languages and standardized protocols marks a crucial step toward enterprise-grade scalability, decoupling tool definition from execution logic to support thousands of heterogeneous APIs.

Despite these architectural advancements, significant hurdles remain in data quality, evaluation rigor, and security. The scarcity of high-quality, diverse training data has spurred the development of multi-agent synthetic data generation frameworks, yet contradictions persist regarding the fidelity of synthetic environments versus real-world applicability. Similarly, evaluation methodologies are evolving from single-turn accuracy metrics to comprehensive benchmarks that assess tool necessity, nested dependencies, and stateful multi-turn coherence. These efforts have exposed a pervasive "knowing-doing gap," where models recognize the need for tools but fail to execute them correctly, highlighting the need for metrics that capture strategic judiciousness rather than mere syntactic correctness. Concurrently, the security landscape has shifted from simple prompt injection to complex, multi-stage attacks targeting the agent-tool interface, including tool poisoning, sequential attack chaining, and resource amplification, necessitating a paradigm shift toward trajectory-level monitoring and attribute-based access control.

Looking forward, the trajectory of tool learning points toward the realization of generalist, self-improving agents capable of operating in open-world settings. Future progress depends on the synthesis of agent-native interface designs, scalable synthetic ecosystems, and robust governance frameworks that ensure economic sustainability and safety. The integration of human-centric pedagogical mechanisms and teachable autonomy will be essential for aligning agent behaviors with user intent while maintaining the flexibility to adapt to unforeseen challenges. Ultimately, the maturation of this field relies on moving beyond isolated function calls to create cohesive, resilient systems that can reason, plan, and act autonomously within complex, interdependent environments, transforming large language models from passive knowledge repositories into active participants in scientific discovery, software engineering, and daily life.